30-Minute Student Task

Python Student Performance Tracker — Solved

Name: DASARI RAM MOHAN

Register Number: 99230040291

Department: CSE

College: Kalasalingam University of Research and Education

1. Task Summary

The task is to create a Python-based Student Performance Tracker. The program accepts a student's name and marks for three subjects, checks whether the entered marks are valid, calculates total and average marks, assigns a performance grade, and prints a clear result.

2. Requirement Checklist

Requirement	How it is implemented
Accept student name	Uses input() and removes unnecessary spaces with strip().
Accept subject marks	Reads three subject marks and converts them to integers.
Validate marks	Accepts only marks from 0 to 100.
Calculate total	Uses sum() to add all subject marks.
Calculate average	Divides total marks by the number of subjects.
Assign performance	Uses if / elif / else conditions based on average.
Display result	Prints the student's name, total, average, and performance grade.

3. Complete Python Solution

```
student_name = input("Enter student name: ").strip()
marks = []

for subject in range(1, 4):
    while True:
        mark = int(input(f"Enter marks for Subject {subject}: "))
        if 0 <= mark <= 100:
            marks.append(mark)
            break
        else:
            print("Please enter a mark between 0 and 100.")

total = sum(marks)
average = total / len(marks)

if average >= 90:
    grade = "Outstanding"
elif average >= 80:
    grade = "Excellent"
elif average >= 70:
    grade = "Very Good"
elif average >= 60:
    grade = "Good"
elif average >= 50:
    grade = "Pass"
else:
    grade = "Needs Improvement"

print("\nStudent Performance")
print("Name:", student_name)
print("Total Marks:", total)
print("Average:", average)
print("Performance:", grade)
```

4. How the Program Works

INPUT: The program receives the student's name and marks for three subjects.

VALIDATION: Every mark is checked so that values outside the 0–100 range are rejected.

PROCESS: The program calculates the total and average using basic Python arithmetic and the sum() function.

PERFORMANCE: Conditional statements classify the average into an understandable performance level.

OUTPUT: The final student name, total, average, and performance level are displayed.

5. Test Cases and Expected Results

Test	Input	Expected Result
Student name	Ananya	Name is displayed correctly
Outstanding marks	95, 94, 96	Outstanding
Excellent marks	84, 86, 82	Excellent
Very good marks	72, 75, 74	Very Good
Pass marks	55, 58, 52	Pass
Invalid marks	110	Program requests a valid mark

6. Sample Demonstration

```
Enter student name: Ananya
Enter marks for Subject 1: 84
Enter marks for Subject 2: 86
Enter marks for Subject 3: 82

Student Performance
Name: Ananya
Total Marks: 252
Average: 84.0
Performance: Excellent
```

7. Advantages of the System

The system is beginner-friendly and demonstrates Python input handling, lists, loops, validation, arithmetic operations, and conditional statements. It reduces manual calculation work and gives a consistent performance classification.

8. Conclusion

The Python Student Performance Tracker combines fundamental Python concepts to solve a practical academic task. It accepts student information, validates marks, calculates performance statistics, and presents the result clearly. The application can later be expanded to support multiple students, more subjects, subject-wise reports, ranking, and file-based storage.